

(12) **United States Patent**
Hart et al.

(10) **Patent No.:** **US 12,406,544 B1**
(45) **Date of Patent:** **Sep. 2, 2025**

(54) **ZERO-TRUST DISTRIBUTED VOTING SYSTEM**

- (71) Applicants: **Brandon Hart**, Jacksonville, FL (US);
Courtney Roach, Jacksonville, FL (US); **Cedric Harris**, Jacksonville, FL (US)
- (72) Inventors: **Brandon Hart**, Jacksonville, FL (US);
Courtney Roach, Jacksonville, FL (US); **Cedric Harris**, Jacksonville, FL (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

(21) Appl. No.: **18/669,861**

(22) Filed: **May 21, 2024**

(51) **Int. Cl.**
G07C 13/00 (2006.01)
H04L 9/00 (2022.01)
H04L 9/08 (2006.01)
H04L 9/32 (2006.01)

(52) **U.S. Cl.**
CPC **G07C 13/00** (2013.01); **H04L 9/0869** (2013.01); **H04L 9/3231** (2013.01); **H04L 9/50** (2022.05)

(58) **Field of Classification Search**
CPC **G07C 13/00**; **H04L 9/50**; **H04L 9/0869**; **H04L 9/3231**
USPC **235/386**
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

2019/0288846	A1 *	9/2019	Zawierka		G06Q 30/00
2020/0258338	A1 *	8/2020	Goswami		H04L 63/12
2023/0082768	A1 *	3/2023	Noe		G07C 13/00
					705/12
2023/0403144	A1 *	12/2023	Rhodin		H04L 9/3239
2024/0244049	A1 *	7/2024	Khrabryi		H04L 63/1466
2025/0014410	A1 *	1/2025	Hwang		G06Q 20/3827

* cited by examiner

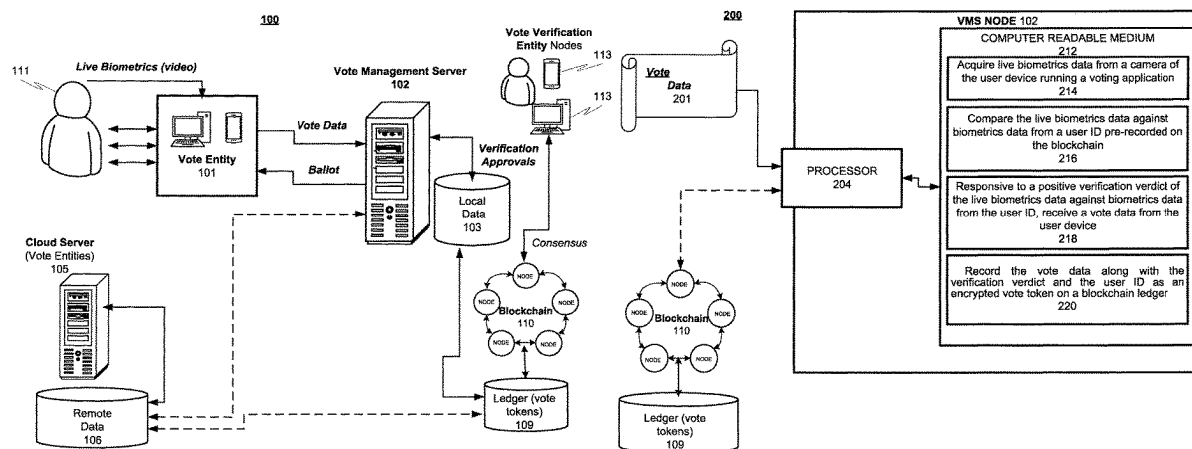
Primary Examiner — Daniel A Hess

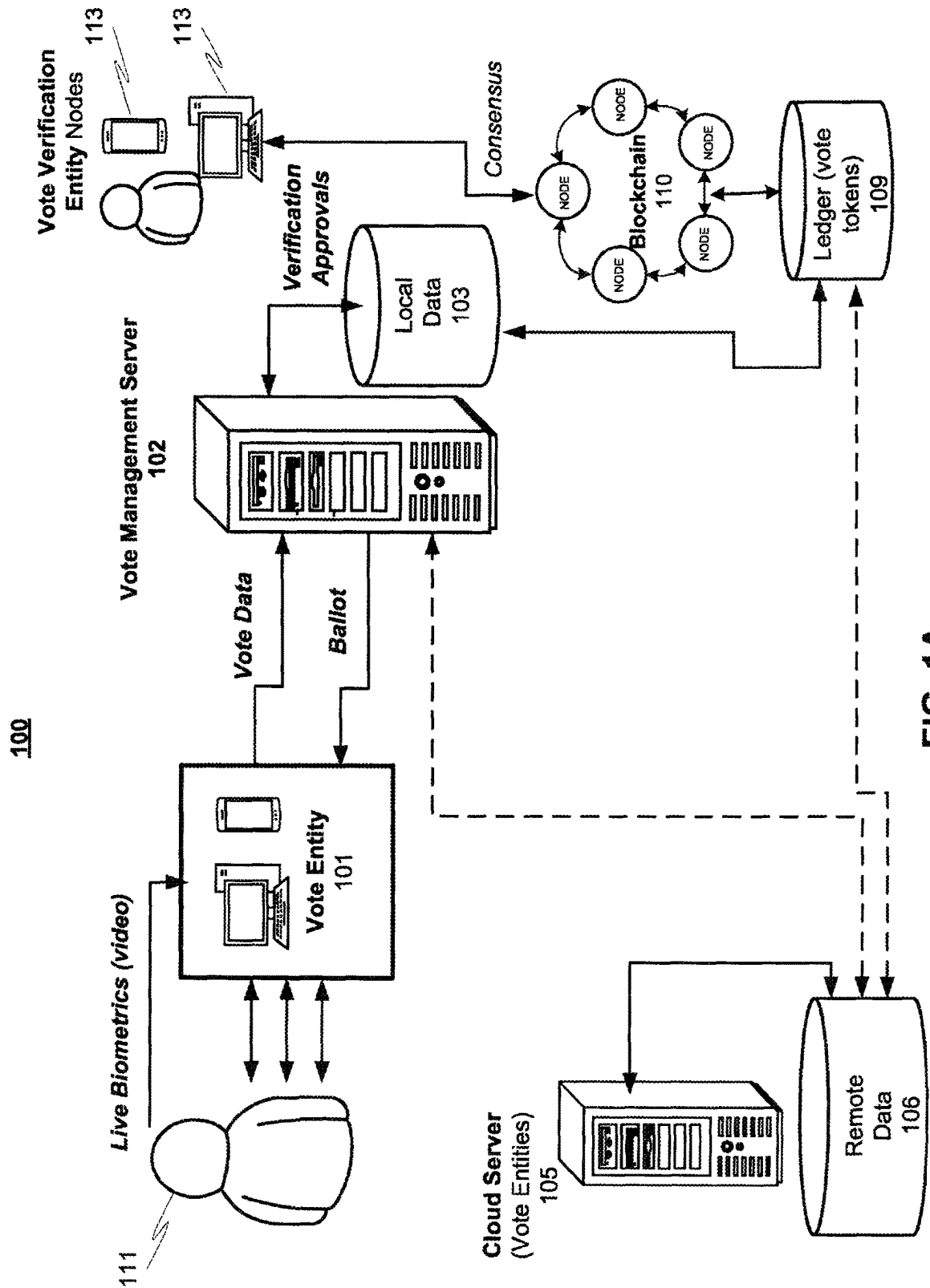
(74) *Attorney, Agent, or Firm* — The Rapacke Law Group, P.A.

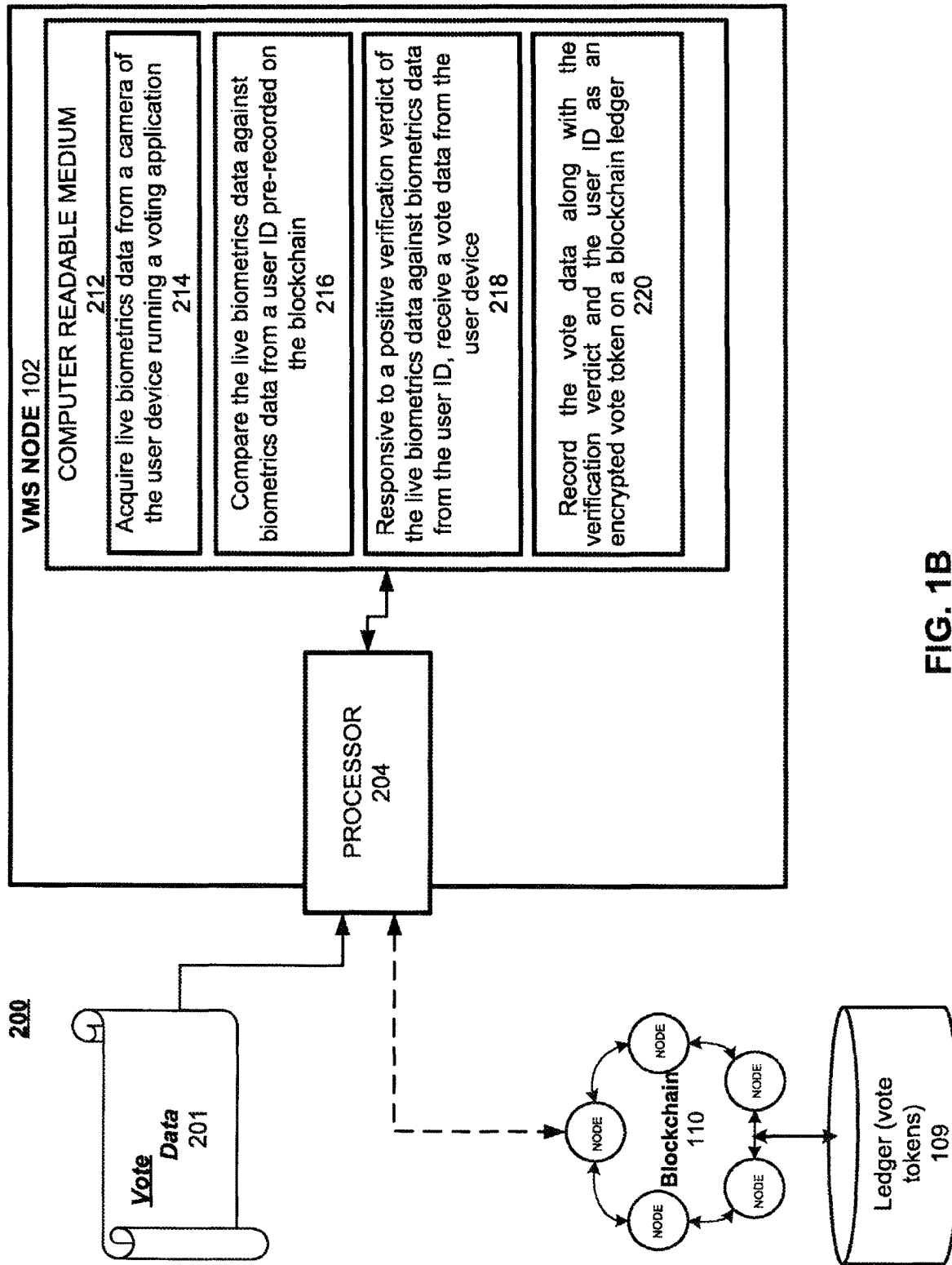
(57) **ABSTRACT**

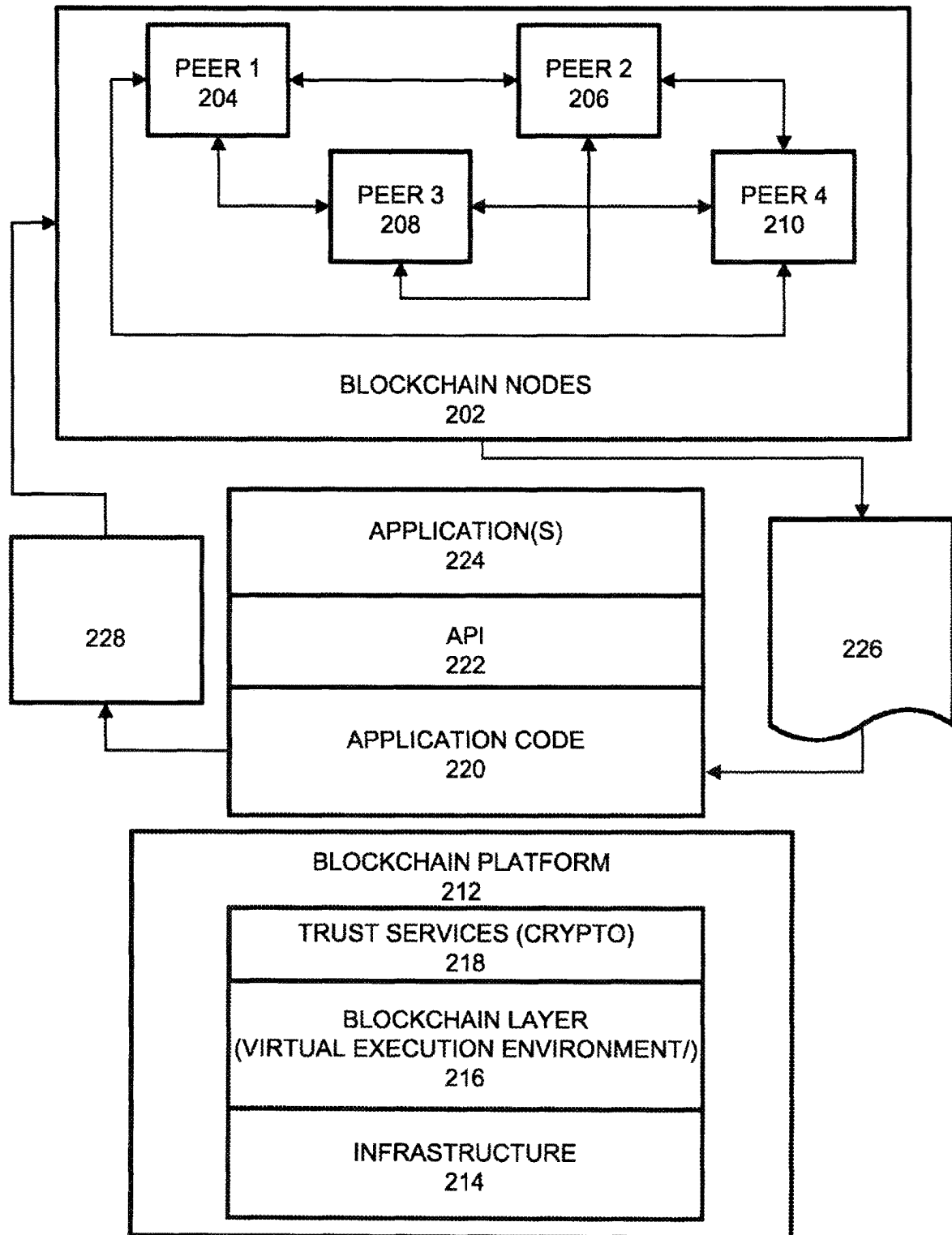
A system for an automated zero-trust distributed voting and vote management based on protected voter-related data, including a processor of a vote management server node connected over a blockchain network to at least one user device onboarded onto the blockchain and a memory on which are stored machine-readable instructions that when executed by the processor, cause the processor to: acquire live biometrics data from a camera of the user device running a voting application; compare the live biometrics data against biometrics data from a user ID pre-recorded on the blockchain; responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, receive a vote data from the user device; and record the vote data along with the verification verdict and the user ID as an encrypted vote token on a blockchain ledger.

20 Claims, 12 Drawing Sheets







200**FIG. 2A**

250

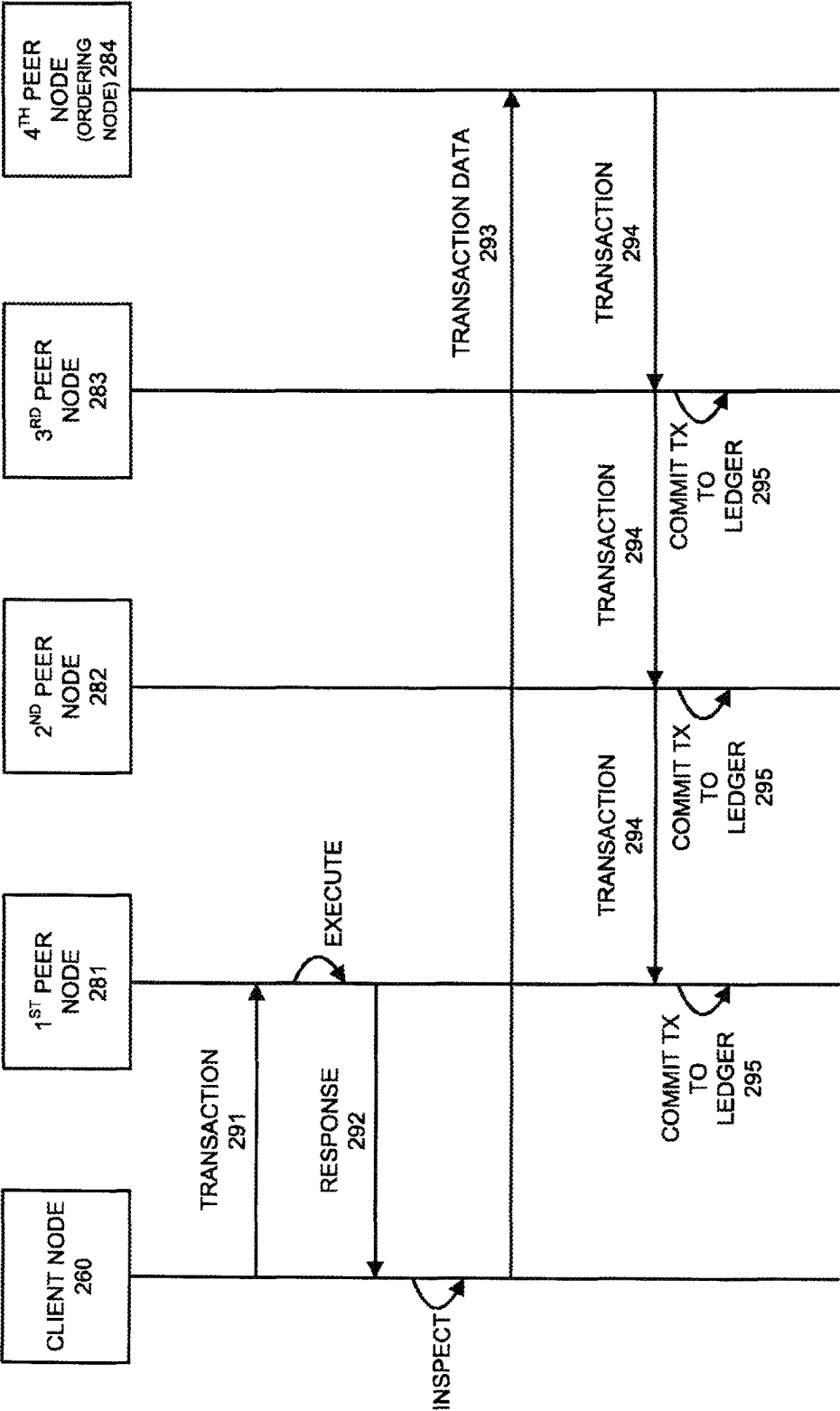


FIG. 2B

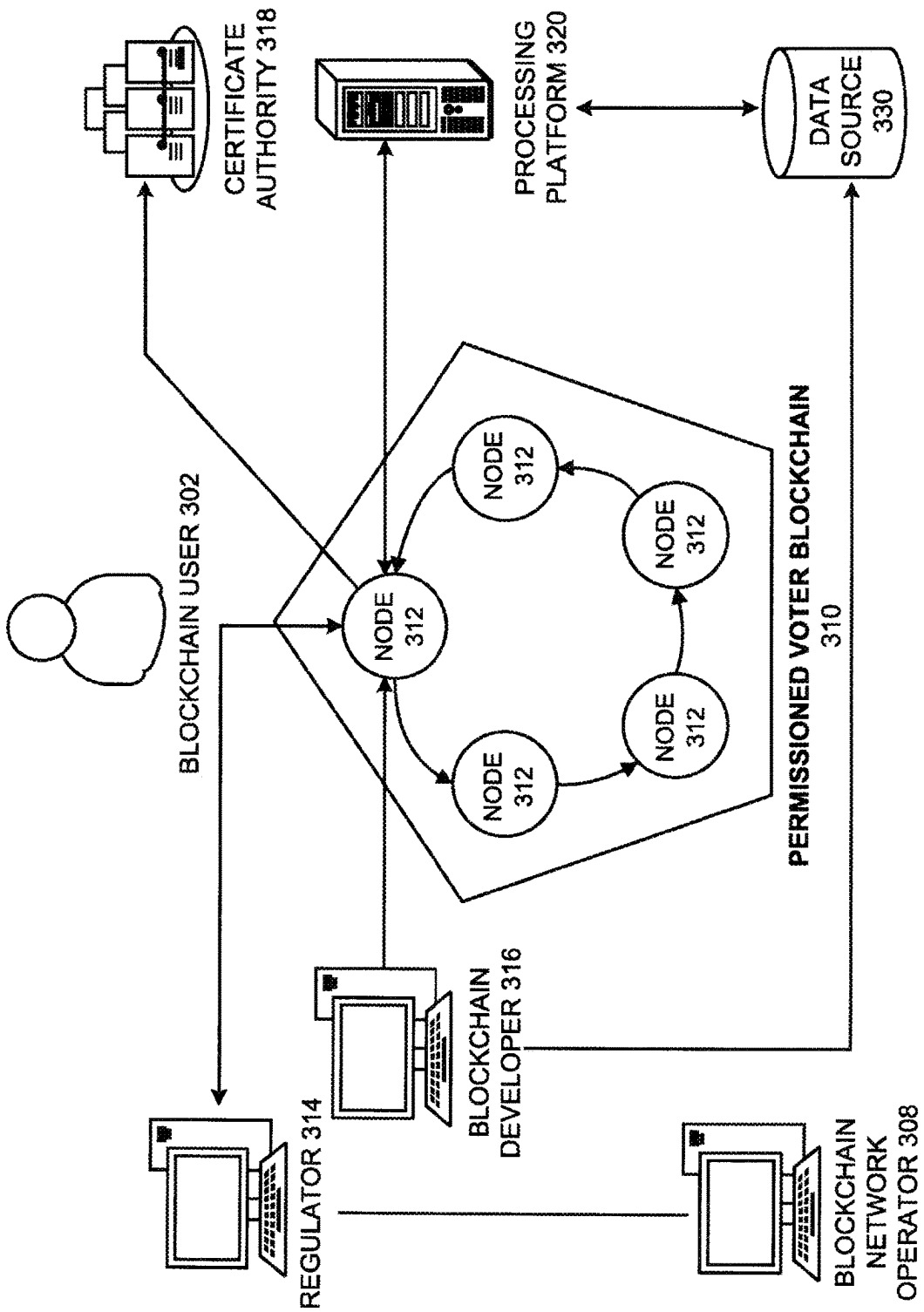
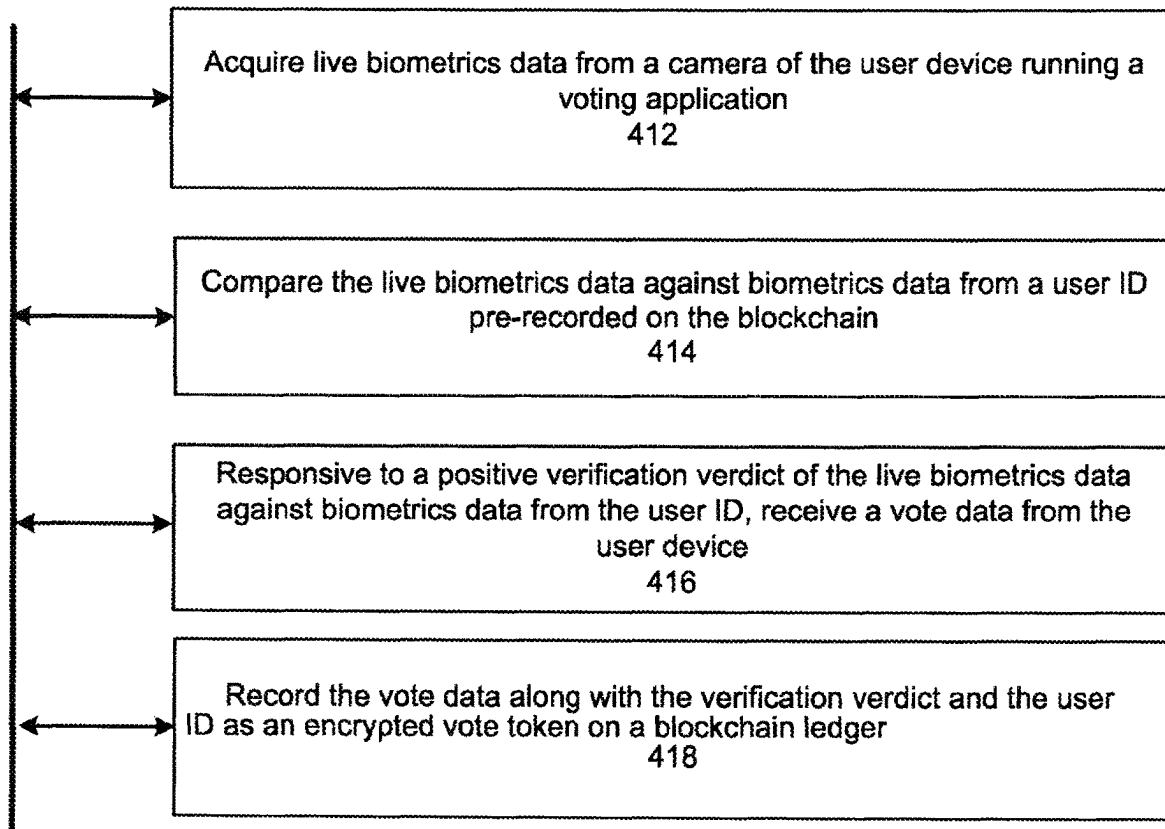
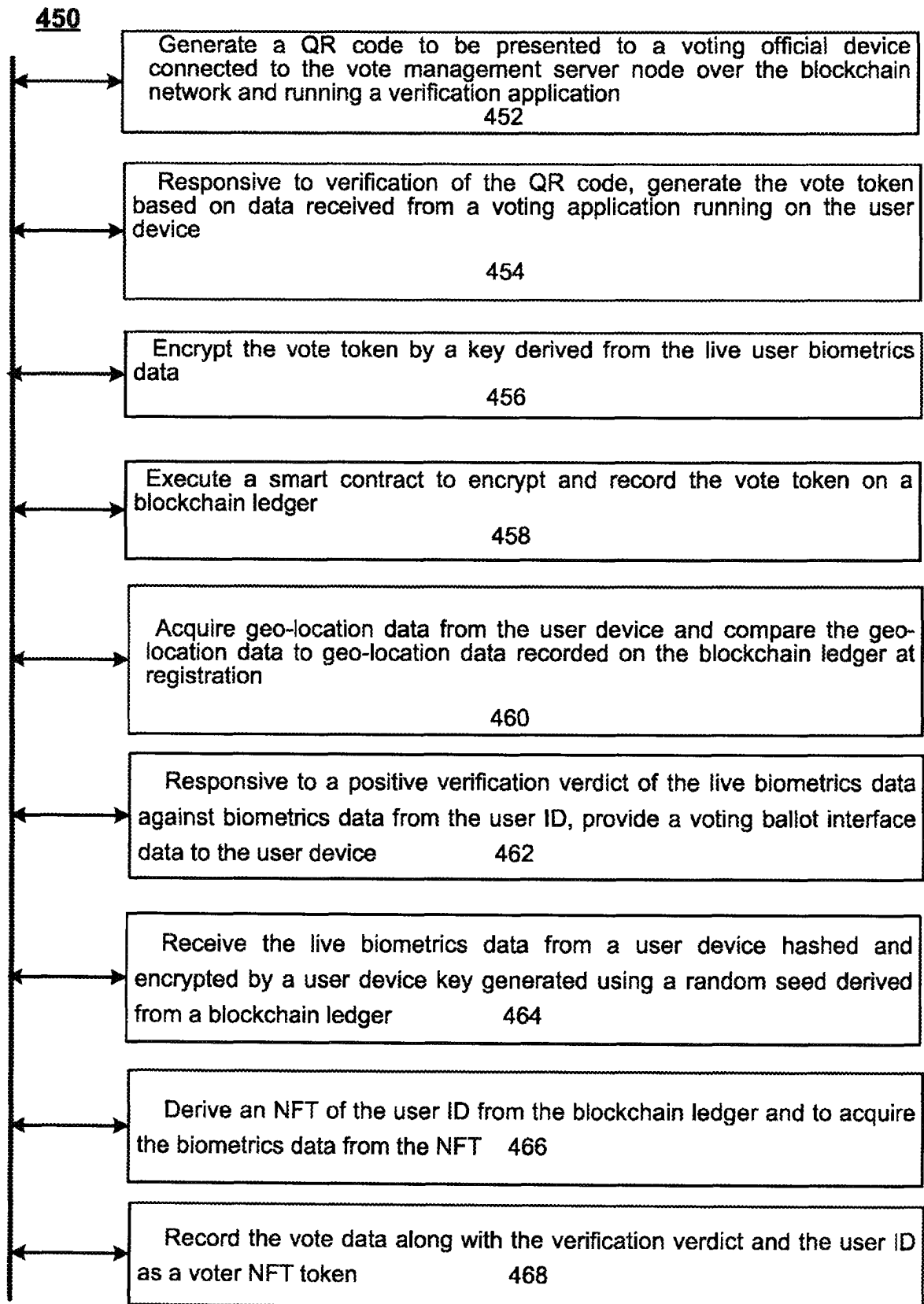
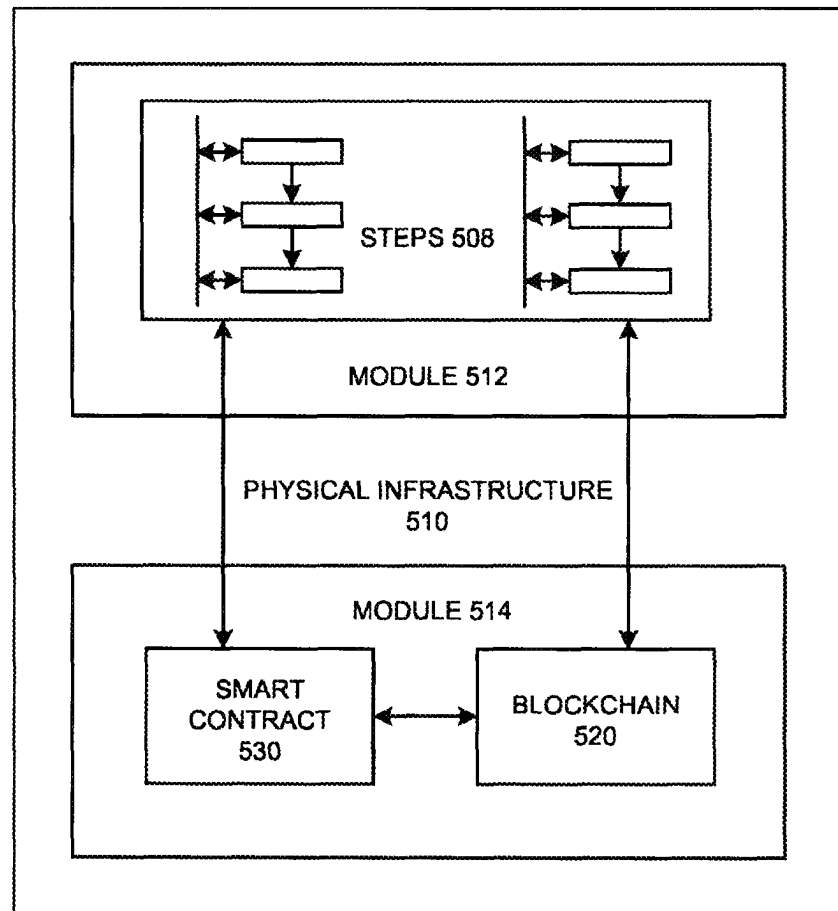
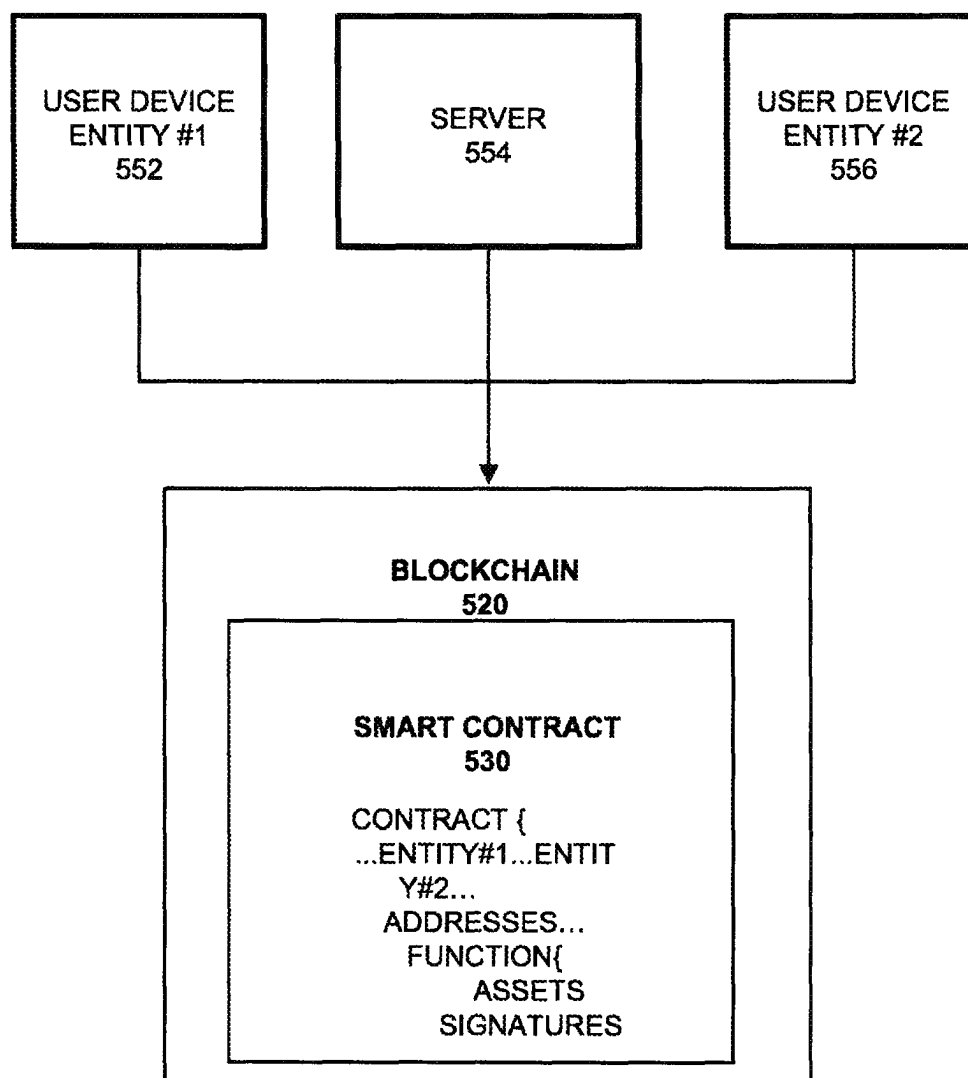


FIG. 3

400**FIG. 4A**

**FIG. 4B**

500**FIG. 5A**

550**FIG. 5B**

600

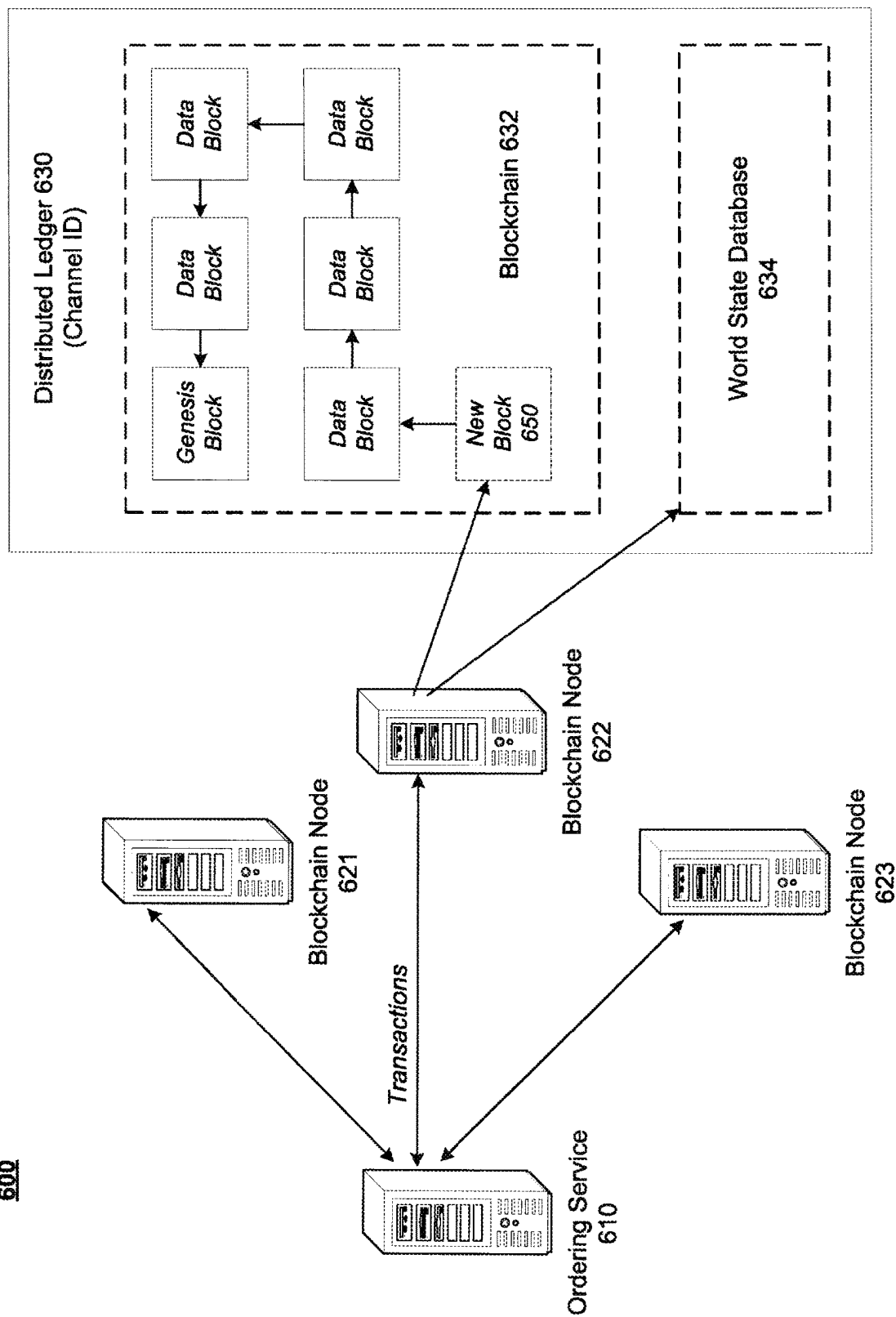


FIG. 6A

Data Block 650

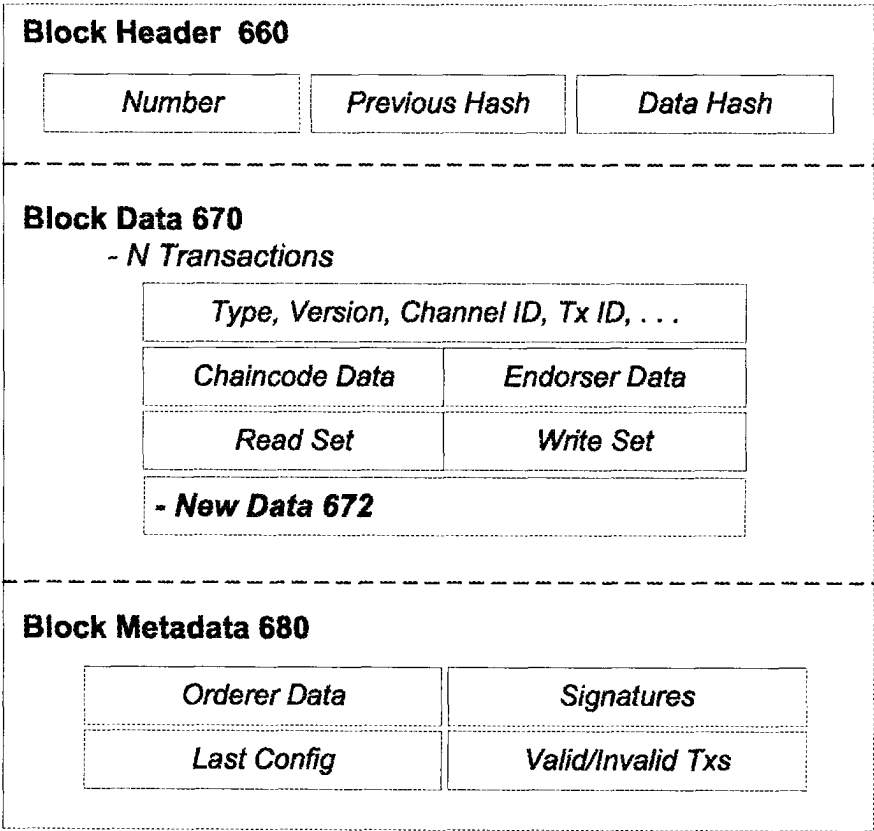


FIG. 6B

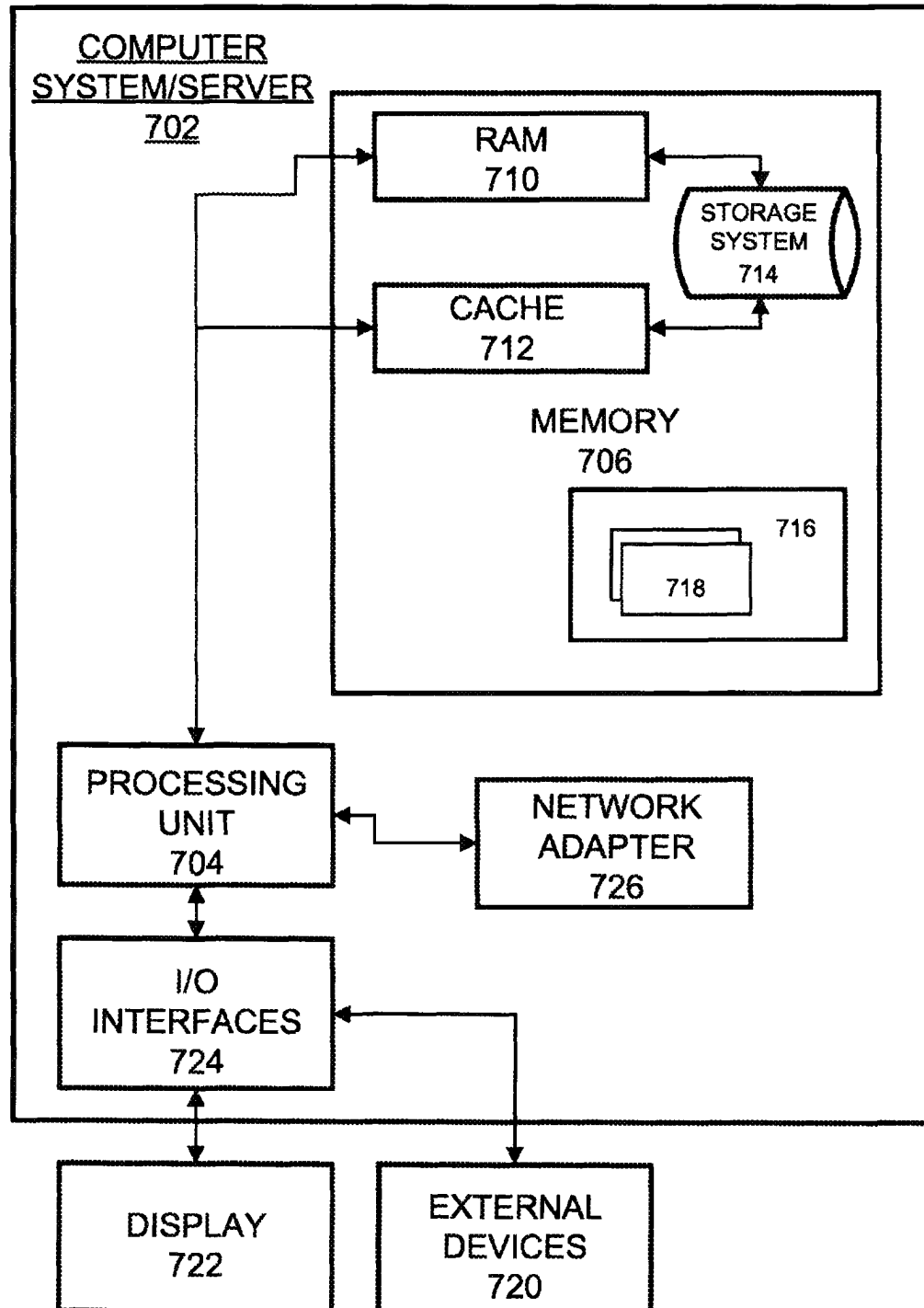
700

FIG. 7

1

ZERO-TRUST DISTRIBUTED VOTING SYSTEM

TECHNICAL FIELD

This application generally relates to a computerized voting system, and more particularly, to a zero-trust distributed voting method and system based on blockchain networks.

BACKGROUND

In recent years, several problems with voting in national and local elections in the United States have emerged. For example, in the 2000 US presidential election, paper balloting revealed problems in determining voter intent when the paper ballots did not clearly reflect voter choice. For example, many disputed ballots had “hanging chads”, i.e., machine readable locations on the paper ballot that should reveal candidate selections but were ambiguous because the paper in the hole was not fully punched through. More recently, the 2020 US presidential election faced controversy on two fronts. First, suspicions had been raised that electronic vote tabulating machines connected to central vote counters over the Internet had back doors that would allow hacking and vote count rigging by bad actors. Second, there was a perceived need to provide alternatives to in-person voting during the COVID-19 pandemic to avoid the risk of voters contracting COVID-19 while standing in line on a single election day. Many states adopted, legally or otherwise, a system of remote drop boxes that contained ballots of voters who opted to vote using mail-in ballots that were handed out freely and sometime unsolicited through the US mail system. Because the US mail system is prone to mis-delivery, the drop boxes were not adequately monitored, and the identities of the voters who filled out the ballots were never witnessed or authenticated by election officials, many citizens became suspicious that ballots were manufactured or stolen and voters’ signatures forged to add fraudulent votes for a particular candidate.

These situations have created controversy and undermined the integrity of the voting process in the US, which is not desirable regardless of candidate preference. What is needed, for the US and other countries with democratically elected officials and ballot resolutions, is a system in which voting is private, authentic, secure, verifiable, and unbackable in order to promote confidence in the election results that is important to the citizens of any self-governed entity. Many remote voting systems use conventional databases.

A centralized database stores and maintains data in one single database (e.g., database server) at one location. This location is often a central computer, for example, a desktop central processing unit (CPU), a server CPU, or a mainframe computer. Information stored on a centralized database is typically accessible from multiple different points. Multiple users or client workstations can work simultaneously on the centralized database, for example, based on a client/server configuration. A centralized database is easy to manage, maintain, and control, especially for purposes of security because of its single location. Within a centralized database, data redundancy is minimized as a single storing place of all data also implies that a given set of data only has one primary record.

However, a centralized database suffers from significant drawbacks. For example, a centralized database has a single point of failure. In particular, if there are no fault-tolerance considerations and a hardware failure occur (for example, the hardware, a firmware, and/or a software failure), all data

2

within the database is lost and work of all users is interrupted. In addition, centralized databases are highly dependent on network connectivity. As a result, the slower the connection, the amount of time needed for each database access is increased. Another drawback is the occurrence of bottlenecks when a centralized database experiences high traffic due to a single location. Furthermore, a centralized database provides limited access to data because only one copy of the data is maintained by the database. As a result, multiple devices cannot access the same piece of data at the same time without creating significant problems or risk overwriting stored data. Furthermore, because a database storage system has minimal to no data redundancy, data that is unexpectedly lost is very difficult to retrieve other than through manual operation from back-up storage.

Conventionally, a centralized database is limited by low search capability, lack of security and slow speed of transactions. As such, what is needed is blockchain-based solution to overcome these significant drawbacks.

Accordingly, it is desired to maintain an immutable provenance of the votes and records of the voting authentication using a blockchain shared immutable ledger.

SUMMARY

One example embodiment provides a system for an automated zero-trust distributed voting and vote management based on protected voter-related data, including a processor of a vote management server node connected over a blockchain network to at least one user device onboarded onto the blockchain and a memory on which are stored machine-readable instructions that when executed by the processor, cause the processor to: acquire live biometrics data from a camera of the user device running a voting application; compare the live biometrics data against biometrics data from a user ID pre-recorded on the blockchain; responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, receive a vote data from the user device; and record the vote data along with the verification verdict and the user ID as an encrypted vote token on a blockchain ledger.

Another example embodiment provides a method that includes one or more of acquiring live biometrics data from a camera of the user device running a voting application; comparing the live biometrics data against biometrics data from a user ID pre-recorded on the blockchain; responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, receiving a vote data from the user device; and recording the vote data along with the verification verdict and the user ID as an encrypted vote token on a blockchain ledger.

A further example embodiment provides a non-transitory computer-readable medium comprising instructions, that when read by a processor, cause the processor to perform one or more of acquiring live biometrics data from a camera of the user device running a voting application; comparing the live biometrics data against biometrics data from a user ID pre-recorded on the blockchain; responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, receiving a vote data from the user device; and recording the vote data along with the verification verdict and the user ID as an encrypted vote token on a blockchain ledger.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1A illustrates a network diagram of a zero-trust distributed voting and vote management system including a blockchain ledger, according to example embodiments.

3

FIG. 1B illustrates an example peer node configuration of a Vote Management Server node, according to example embodiments.

FIG. 2A illustrates a blockchain peer node configuration, according to example embodiments.

FIG. 2B illustrates execution of transactions associated with digital asset representations of the votes and voter data, according to example embodiments.

FIG. 3 illustrates a permissioned network, according to example embodiments.

FIG. 4A illustrates a flow diagram of a method for zero-trust distributed voting and vote management system, according to example embodiments.

FIG. 4B illustrates a further flow diagram of the method for zero-trust distributed voting and vote management system, according to example embodiments.

FIG. 5A illustrates an example system configured to perform one or more operations described herein, according to example embodiments.

FIG. 5B illustrates a smart contract configuration among contracting parties and a mediating server configured to enforce the smart contract terms on the blockchain according to example embodiments.

FIG. 6A illustrates a process of new data being added to a blockchain ledger, according to example embodiments.

FIG. 6B illustrates contents of a data block including the new data, according to example embodiments.

FIG. 7 illustrates an example computer/server system that supports one or more of the example embodiments.

DETAILED DESCRIPTION

It will be readily understood that the instant components, as generally described and illustrated in the figures herein, may be arranged and designed in a wide variety of different configurations. Thus, the following detailed description of the embodiments of at least one of a method, apparatus, non-transitory computer readable medium and system, as represented in the attached figures, is not intended to limit the scope of the application as claimed but is merely representative of selected embodiments.

The instant features, structures, or characteristics as described throughout this specification may be combined in any suitable manner in one or more embodiments. For example, the usage of the phrases “example embodiments”, “some embodiments”, or other similar language, throughout this specification refers to the fact that a particular feature, structure, or characteristic described in connection with the embodiment may be included in at least one embodiment. Thus, appearances of the phrases “example embodiments”, “in some embodiments”, “in other embodiments”, or other similar language, throughout this specification do not necessarily all refer to the same group of embodiments, and the described features, structures, or characteristics may be combined in any suitable manner in one or more embodiments.

In addition, while the term “message” may have been used in the description of embodiments, the application may be applied to many types of network data, such as, packet, frame, datagram, etc. The term “message” also includes packet, frame, datagram, and any equivalents thereof. Furthermore, while certain types of messages and signaling may be depicted in exemplary embodiments they are not limited to a certain type of message, and the application is not limited to a certain type of signaling.

Example embodiments provide methods, systems, components, non-transitory computer readable media, devices,

4

and/or networks, which provide for secure execution of voting process based on blockchain networks.

A decentralized database is a distributed storage system which includes multiple nodes that communicate with each other. A blockchain is an example of a decentralized database which includes an append-only immutable data structure resembling a distributed ledger capable of maintaining records between mutually untrusted parties. The untrusted parties are referred to herein as peers or peer nodes. Each peer maintains a copy of the database records and no single peer can modify the database records without a consensus being reached among the distributed peers. For example, the peers may execute a consensus protocol to validate blockchain storage transactions, group the storage transactions into blocks, and build a hash chain over the blocks. This process forms the ledger by ordering the storage transactions, as is necessary, for consistency. In a public or permission-less blockchain, anyone can participate without a specific identity. Public blockchains often involve native crypto-currency or other assets and use consensus based on various protocols such as Proof of Work (PoW). On the other hand, a permissioned blockchain used in the disclosed embodiments provides a system which can secure interactions among a group of entities which share a common goal, but which do not fully trust one another, such as businesses that exchange funds, goods, information, and the voting records in this case.

A blockchain operates arbitrary, programmable logic, tailored to a decentralized storage scheme and referred to as “smart contracts” or “chaincodes.” In some cases, specialized chaincodes may exist for management functions and parameters which are referred to as system chaincode. Smart contracts are trusted distributed applications which leverage tamper-proof properties of the blockchain and an underlying agreement between nodes which is referred to as an endorsement or endorsement policy. In general, blockchain transactions typically must be “endorsed” before being committed to the blockchain while transactions which are not endorsed are disregarded. A typical endorsement policy allows chaincode to specify endorsers for a transaction in the form of a set of peer nodes that are necessary for endorsement. When a client sends the transaction to the peers specified in the endorsement policy, the transaction is executed to validate the transaction. After validation, the transactions enter an ordering phase in which a consensus protocol is used to produce an ordered sequence of endorsed transactions grouped into blocks.

Nodes are the communication entities of the blockchain system. A “node” may perform a logical function in the sense that multiple nodes of different types can run on the same physical server. Nodes are grouped into trust domains and are associated with logical entities that control them in various ways. Nodes may include different types, such as a client or submitting-client node which submits a transaction-invocation to an endorser (e.g., peer), and broadcasts transaction-proposals to an ordering service (e.g., ordering node). Another type of node is a peer node which can receive client submitted transactions, commit the transactions and maintain a state and a copy of the ledger of blockchain transactions. Peers can also have the role of an endorser, although it is not a requirement. An ordering-service-node or an orderer is a node running the communication service for all nodes, and which implements a delivery guarantee, such as a broadcast to each of the peer nodes in the system when committing transactions and modifying a world state of the

blockchain, which is another name for the initial blockchain transaction which normally includes control and setup information.

A ledger is a sequenced, tamper-resistant record of all state transitions of a blockchain. State transitions may result from chaincode invocations (i.e., transactions) submitted by participating parties (e.g., client nodes, ordering nodes, endorser nodes, peer nodes, etc.). A transaction may result in a set of asset key-value pairs being committed to the ledger as one or more operands, such as creates, updates, deletes, and the like. The ledger includes a blockchain (also referred to as a chain), which is used to store an immutable, sequenced record in blocks. The ledger also includes a state database which maintains a current state of the blockchain. There is typically one ledger per channel. Each peer node maintains a copy of the ledger for each channel of which they are a member.

A chain is a transaction log which is structured as hash-linked blocks, and each block contains a sequence of N transactions where N is equal to or greater than one. The block header includes a hash of the block's transactions, as well as a hash of the prior block's header. In this way, all transactions on the ledger may be sequenced and cryptographically linked together. Accordingly, it is not possible to tamper with the ledger data without breaking the hash links. A hash of a most recently added blockchain block represents every transaction on the chain that has come before it, making it possible to ensure that all peer nodes are in a consistent and trusted state. The chain may be stored on a peer node file system (i.e., local, attached storage, cloud, etc.), efficiently supporting the append-only nature of the blockchain workload.

The current state of the immutable ledger represents the latest values for all keys that are included in the chain transaction log. Because the current state represents the latest key values known to a channel, it is sometimes referred to as a world state. Chaincode invocations execute transactions against the current state data of the ledger. To make these chaincode interactions efficient, the latest values of the keys may be stored in a state database. The state database may be simply an indexed view into the chain's transaction log, it can therefore be regenerated from the chain at any time. The state database may automatically be recovered (or generated if needed) upon peer node startup, and before transactions are accepted.

Some benefits of the instant solutions described and depicted herein include a method and system for monitoring of batteries in blockchain networks. The exemplary embodiments solve the issues of time and trust by extending features of a regular database such as immutability, digital signatures and being a single source of truth. The exemplary embodiments provide a solution for monitoring, verifying and recording votes in blockchain-based network. The blockchain networks may be homogenous based on the asset type and rules that govern the assets based on the smart contracts.

Blockchain employed in the disclosed zero-trust distributed voting method and system is different from a traditional database in that blockchain is not a central storage, but rather a decentralized, immutable, and secure storage, where nodes must share in changes to records in the storage. Some properties that are inherent in blockchain and which help to implement the blockchain include, but are not limited to, an immutable ledger, smart contracts, security, privacy, decentralization, consensus, endorsement, accessibility, and the like, which are further described herein. According to various aspects, the system for zero-trust distributed voting in

blockchain networks is implemented due to immutable accountability, security, privacy, permitted decentralization, availability of smart contracts, endorsements and accessibility that are inherent and unique to blockchain. In particular, the blockchain ledger data is immutable and that provides for efficient method for zero-trust distributed voting in blockchain networks. Also, use of the encryption in the blockchain provides security and builds trust. The smart contract manages the state of the asset to complete the life-cycle. The example blockchains are permission decentralized. Thus, each end user may have its own ledger copy to access. Multiple organizations (and peers such as Post Office, Sheriff's Office, Voting Auditing Authorities, etc.) may be on-boarded onto the blockchain network. The key organizations may serve as endorsing peers to validate the smart contract execution results, read-set and write-set. In other words, the blockchain inherent features provide for efficient implementation of zero-trust distributed voting method and system.

One of the benefits of the example embodiments is that it improves the functionality of a computing system by implementing a method for zero-trust distributed voting in blockchain-based systems. Through the blockchain system described herein, a computing system can perform functionality for zero-trust distributed voting method in blockchain networks by providing access to capabilities such as distributed ledger, peers, encryption technologies, MSP, event handling, etc. Also, the blockchain enables to create a business network and make any users or organizations to on-board for participation. As such, the blockchain is not just a database. The blockchain comes with capabilities to create a Business Network of users and on-board/off-board organizations such as voting organizations or departments within a voting facility collaborate and execute service processes in the form of smart contracts.

The example embodiments provide numerous benefits over a traditional database. For example, through the blockchain the embodiments provide for immutable accountability, security, privacy, permitted decentralization, availability of smart contracts, endorsements and accessibility that are inherent and unique to the blockchain.

Meanwhile, a traditional database could not be used to implement the example embodiments because it does not bring all parties on the business network, it does not create trusted collaboration and does not provide for an efficient storage of digital assets associated with the zero-trust distributed voting. The traditional database does not provide for a tamper proof storage and does not provide for preservation of the digital assets being stored. Thus, the proposed method for zero-trust distributed voting in blockchain networks cannot be implemented in the traditional database.

Meanwhile, if a traditional database were to be used to implement the example embodiments, the example embodiments would have suffered from unnecessary drawbacks such as search capability, lack of security and slow speed of transactions. Additionally, the automated method for zero-trust distributed voting in the blockchain network would simply not be possible.

Accordingly, the example embodiments provide for a specific solution to a problem in the arts/field of zero-trust distributed voting in the blockchain networks.

The example embodiments also change how data may be stored within a block structure of the blockchain. For example, a digital asset representing a voter ID, a vote or vote-related data may be securely stored within a certain portion of the data block (i.e., within header, data segment, or metadata). By storing the digital asset data within the data

blocks of a blockchain, the digital asset data may be appended to an immutable blockchain ledger through a hash-linked chain of blocks. In some embodiments, the data block may be different than a traditional data block by having a personal data associated with the digital asset not stored together with the assets within a traditional block structure of a blockchain. By removing the personal data associated with the digital asset, the blockchain can provide the benefit of anonymity based on immutable accountability and security.

According to the exemplary embodiments, a blockchain may be used for creating a shared immutable ledger that records the various events during the lifecycle of a voter record and may make relevant subsets of the data to various interested parties participating in a blockchain network. Details about the voter, voter identification and verification, vote, date/time of vote, location of vote, voter registration, voting authority (location or department, etc.) may be collected and stored on the blockchain ledger. Other information related to the trusted origination of the vote may be recorded as well. In one embodiment, the voter data may include biometric data captured by the voter or verifier device itself or data that is communicated to the vote management server node by an external source, such as a vote management entity that monitors vote-related transactions and voter verifications.

According to one embodiment, the blockchain may be used to capture the various events encountered by the voter verification data and the vote data during its flow lifecycle. The data collected during the access of the voter-related data may be recorded on the blockchain ledger. Having the vote casting/transactions data in the blockchain provides for immutability and verifiability of the data, which are the key requirements in a multi-party system.

The data residing on the blockchain may be shared among selected parties via secure channels. For example, some data may be shared only between the vote verifiers and managers, other data may be shared only between the voter and voter representative, yet other data may be shared between the vote auditors, official observers, other officials and so on.

FIG. 1A illustrates a logic network diagram for zero-trust distributed voting and vote management system using a blockchain network, according to example embodiments.

Referring to FIG. 1A, the process can be defined by a number of interactions between certain entities and a number of steps that define the process. The following entities described below are involved in the process. In one embodiment, blockchain servers may be implemented as a network of computers that operate as a private blockchain, and can independently verify the authenticity of a vote using an instantaneous, chain-of-events consensus described in the co-pending US Patent Pub. No. 2023/0008302.

In one embodiment, a zero-trust application is provided that allows the user to create secure containers or “vaults” whose decryption keys are not susceptible to hacking through breach of encryption keys stored in a single location, allowing the owner of the data to maintain control of his or her own data. A zero-trust cryptographic system suitable for use in creating secure containers is described in the co-pending U.S. Pat. No. 11,570,155.

Referring to FIG. 1A, the example network **100** includes a Vote Management Server (VMS) node **102** connected to a cloud server node(s) **105** over a network. The VMS node **102** may be configured to host a local database of voter-related data. The VMS node **102** may receive a vote data from the user device entity **101** used by a voting user **111**. The VMS node **102** may receive live biometrics data from

the user device entity **101** to verify the voting user **111** against pre-store user ID or data derived from the user ID (i.e., an official government issued credentials such as driver license, passport, etc.). Note that other forms of biometrics (e.g., voice, iris, fingerprints, etc.) may be used provided that the biometric samples of the voting user **111** are stored at the registration either with the ID or separately.

The VMS node **102** may have access to remote voting user data **106** from a remote database residing on a cloud server **105**. The remote voting user data **106** may be collected from other voting facilities.

In one embodiment, the VMS node **102** may record the vote data on a permissioned blockchain **110** ledger **109**. The VMS node **102** may process vote data including a marked voting bulletin in a digital form by transferring digital assets associated with the voting bulletin marked by the voter to the voter's digital wallet over the blockchain **110** based on a consensus from the vote verification entity nodes **113** confirming, for example, the votes and certifying the transactions. Additionally, confidential voter user personal data may be acquired from the permissioned blockchain **110** in a form of an NFT and may be rendered to the vote verification entity nodes **113** via a management application interface. Once the voter **111** biometrics are verified (based on consensus), a voting ballot may be provided by the VMS node **102** to be rendered on the user device entity **101**.

In this implementation the vote verification entity nodes **113** and the VMS node **102**, and the cloud server **105** may serve as blockchain **110** peer nodes. In one embodiment, local data **103** and remote data **106** may be duplicated on the blockchain ledger **109** for higher security of storage.

FIG. 1B illustrates an example peer node configuration of a Vote Management Server (VMS) node, according to example embodiments.

Referring to FIG. 1B, the example network **200** includes the VMS node **102** connected to the user device entity **101** (see FIG. 1A) to receive the vote data **201**. As discussed above with respect to FIG. 1A, the VMS node **102** may be implemented as a peer on the blockchain **110**.

While this example describes in detail only one VMS node **102**, multiple such nodes may be connected to the network and to the blockchain **110**. It should be understood that the VMS node **102** may include additional components and that some of the components described herein may be removed and/or modified without departing from a scope of the VMS node **102** disclosed herein. The VMS node **102** may be a computing device or a server computer, or the like, and may include a processor, which may be a semiconductor-based microprocessor, a central processing unit (CPU), an application specific integrated circuit (ASIC), a field-programmable gate array (FPGA), and/or another hardware device. Although a single processor is depicted, it should be understood that the VMS node **102** may include multiple processors, multiple cores, or the like, without departing from the scope of the VMS node **102** system.

The VMS node **102** may also include a non-transitory computer readable medium **212** that may have stored thereon machine-readable instructions executable by the processor. Examples of the machine-readable instructions are shown as **214-220** and are further discussed below. Examples of the non-transitory computer readable medium **212** may include an electronic, magnetic, optical, or other physical storage device that contains or stores executable instructions. For example, the non-transitory computer readable medium **212** may be a Random-Access memory

(RAM), an Electrically Erasable Programmable Read-Only Memory (EEPROM), a hard disk, an optical disc, or other type of storage device.

The processor may fetch, decode, and execute the machine-readable instructions **214** to acquire live biometrics data from a camera of the user device **101** running a voting application. The processor may fetch, decode, and execute the machine-readable instructions **216** to compare the live biometrics data against biometrics data from a user ID pre-recorded on the blockchain **110**. The processor may fetch, decode, and execute the machine-readable instructions **218** to responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, receive a vote data from the user device **101**. The processor may fetch, decode, and execute the machine-readable instructions **220** to record the vote data along with the verification verdict and the user ID as an encrypted vote token on a blockchain ledger.

The permissioned blockchain **110** may be configured to use one or more smart contracts that manage transactions for multiple participating nodes and for recording the transactions on the ledger **109**. As discussed above local datasets **103** and **106** may be recorded or duplicated on a private (permissioned) blockchain **110**. This provides a tamper-evident log of votes and voter verifications, enhancing security and transparency.

FIG. 2A illustrates a blockchain architecture configuration **200**, according to example embodiments. Referring to FIG. 2A, the blockchain architecture **200** may include certain blockchain elements, for example, a group of blockchain nodes **202**. The blockchain nodes **202** may include one or more nodes **-210** (these four nodes are depicted by example only). These nodes participate in a number of activities, such as blockchain transaction addition and validation process (consensus). One or more of the blockchain nodes **-210** may endorse transactions based on endorsement policy and may provide an ordering service for all blockchain nodes in the architecture **200**. A blockchain node may initiate a blockchain authentication and seek to write to a blockchain immutable ledger stored in blockchain layer **216**, a copy of which may also be stored on the underpinning physical infrastructure **214**. The blockchain configuration may include one or more applications **224** which are linked to application programming interfaces (APIs) **222** to access and execute stored program/application code **220** (e.g., chaincode, smart contracts, etc.) which can be created according to a customized configuration sought by participants and can maintain their own state, control their own assets, and receive external information. This can be deployed as a transaction and installed, via appending to the distributed ledger, on all blockchain nodes **-210**.

The blockchain base or platform **212** may include various layers of blockchain data, services (e.g., cryptographic trust services, virtual execution environment, etc.), and underpinning physical computer infrastructure that may be used to receive and store new transactions and provide access to auditors which are seeking to access data entries. The blockchain layer **216** may expose an interface that provides access to the virtual execution environment necessary to process the program code and engage the physical infrastructure **214**. Cryptographic trust services **218** may be used to verify transactions such as asset exchange transactions and keep information private.

The blockchain architecture configuration of FIG. 2A may process and execute program/application code **220** via one or more interfaces exposed, and services provided, by blockchain platform **212**. The code **220** may control blockchain

assets. For example, the code **220** can store and transfer data, and may be executed by nodes **-210** in the form of a smart contract and associated chaincode with conditions or other code elements subject to its execution. As a non-limiting example, smart contracts may be created to execute reminders, updates, and/or other notifications subject to the changes, updates, etc. The smart contracts can themselves be used to identify rules associated with authorization and access requirements and usage of the ledger. For example, the voter and/or vote-related information **226** may be processed by one or more processing entities (e.g., virtual machines) included in the blockchain layer **216**. The result **228** may include data blocks reflecting a latest voter and vote-related data including a voter NFT or digital token. The physical infrastructure **214** may be utilized to retrieve any of the data or information described herein.

Within chaincode, a smart contract may be created via a high-level application and programming language, and then written to a block in the blockchain. The smart contract may include executable code which is registered, stored, and/or replicated with a blockchain (e.g., distributed network of blockchain peers). A transaction is an execution of the smart contract code which can be performed in response to conditions associated with the smart contract being satisfied. The executing of the smart contract may trigger a trusted modification(s) to a state of a digital blockchain ledger. The modification(s) to the blockchain ledger caused by the smart contract execution may be automatically replicated throughout the distributed network of blockchain peers through one or more consensus protocols.

The smart contract may write data to the blockchain in the format of key-value pairs. Furthermore, the smart contract code can read the values stored in a blockchain and use them in application operations. The smart contract code can write the output of various logic operations into the blockchain. The code may be used to create a temporary data structure in a virtual machine or other computing platform. Data written to the blockchain can be public and/or can be encrypted and maintained as private. The temporary data that is used/generated by the smart contract is held in memory by the supplied execution environment and then deleted once the data needed for the blockchain is identified.

A chaincode may include the code interpretation of a smart contract, with additional features. As described herein, the chaincode may be program code deployed on a computing network, where it is executed and validated by chain validators together during a consensus process. The chaincode receives a hash and retrieves from the blockchain a hash associated with the data template created by use of a previously stored feature extractor. If the hashes of the hash identifier and the hash created from the stored identifier template data match, then the chaincode sends an authorization key to the requested service. The chaincode may write to the blockchain data associated with the cryptographic details. In FIG. 2A, recordation of vote-related data and voter-related data may include execution of the smart contract. One function may be to commit a transaction related to execution of the smart contract on the ledger for recording current vote data with a timestamp, which may be provided to one or more of the nodes **-210**.

FIG. 2B illustrates an example of a transactional flow **250** between nodes of the blockchain in accordance with an example embodiment. Referring to FIG. 2B, the transaction flow may include a transaction proposal **291** sent by an application client node **260** to an endorsing peer node **281**. The endorsing peer **281** may verify the client signature and execute a chaincode function to initiate the transaction. The

11

output may include the chaincode results, a set of key/value versions that were read in the chaincode (read set), and the set of keys/values that were written in chaincode (write set). The proposal response **292** is sent back to the client **260** along with an endorsement signature, if approved. The client **260** assembles the endorsements into a transaction payload **293** and broadcasts it to an ordering service node **284**. The ordering service node **284** then delivers ordered transactions as blocks to all peers **281-283** on a channel. Before committing to the blockchain, each peer **281-283** may validate the transaction. For example, the peers may check the endorsement policy to ensure that the correct allotment of the specified peers have signed the results and authenticated the signatures against the transaction payload **293**.

Referring again to FIG. 2B, the client node **260** initiates the transaction **291** by constructing and sending a request to the peer node **281**, which is an endorser. The client **260** may include an application leveraging a supported software development kit (SDK), such as NODE, JAVA, PYTHON, and the like, which utilizes an available API to generate a transaction proposal. The proposal is a request to invoke a chaincode function so that data can be read and/or written to the ledger (i.e., write new key value pairs for the assets). The SDK may serve as a shim to package the transaction proposal into a properly architected format (e.g., protocol buffer over a remote procedure call (RPC)) and take the client's cryptographic credentials to produce a unique signature for the transaction proposal.

In response, the endorsing peer node **281** may verify (a) that the transaction proposal is well formed, (b) the transaction has not been submitted already in the past (replay-attack protection), (c) the signature is valid, and (d) that the submitter (client **260**, in the example) is properly authorized to perform the proposed operation on that channel. The endorsing peer node **281** may take the transaction proposal inputs as arguments to the invoked chaincode function. The chaincode is then executed against a current state database to produce transaction results including a response value, read set, and write set. However, no updates are made to the ledger at this point. In **292**, the set of values along with the endorsing peer node's **281** signature is passed back as a proposal response **292** to the SDK of the client **260** which parses the payload for the application to consume.

In response, the application of the client **260** inspects/verifies the endorsing peers' signatures and compares the proposal responses to determine if the proposal response is the same. If the chaincode only queried the ledger, the application would inspect the query response and would typically not submit the transaction to the ordering node service **284**. If the client application intends to submit the transaction to the ordering node service **284** to update the ledger, the application determines if the specified endorsement policy has been fulfilled before submitting (i.e., did all peer nodes necessary for the transaction endorse the transaction). Here, the client may include only one of multiple parties to the transaction. In this case, each client may have their own endorsing node, and each endorsing node will need to endorse the transaction. The architecture is such that even if an application selects not to inspect responses or otherwise forwards an unendorsed transaction, the endorsement policy will still be enforced by peers and upheld at the commit validation phase.

After successful inspection, in step **293** the client **260** assembles endorsements into a transaction and broadcasts the transaction proposal and response within a transaction message to the ordering node **284**. The transaction may contain the read/write sets, the endorsing peers' signatures

12

and a channel ID. The ordering node **284** does not need to inspect the entire content of a transaction in order to perform its operation. Instead, the ordering node **284** may simply receive transactions from all channels in the network, order them chronologically by channel, and create blocks of transactions per channel.

The blocks of the transaction are delivered from the ordering node **284** to all peer nodes **281-283** on the channel. The transactions **294** within the block are validated to ensure any endorsement policy is fulfilled and to ensure that there have been no changes to ledger state for read set variables since the read set was generated by the transaction execution. Transactions in the block are tagged as being valid or invalid. Furthermore, in step **295** each peer node **281-283** appends the block to the channel's chain, and for each valid transaction the write sets are committed to current state database. An event is emitted, to notify the client application that the transaction (invocation) has been immutably appended to the chain, as well as to notify whether the transaction was validated or invalidated.

FIG. 3 illustrates an example of a permissioned blockchain network **300**, which features a distributed, decentralized peer-to-peer architecture, and a certificate authority **318** managing user roles and permissions. In this example, the blockchain user **302** may submit a transaction to the permissioned voter blockchain network **310**. In this example, the transaction can be a deploy, invoke or query, and may be issued through a client-side application leveraging an SDK, directly through a REST API, or the like. Trusted business networks may provide access to regulator systems **314**, such as vote auditors. Meanwhile, a blockchain network operator system of nodes **308** manages member permissions, such as enrolling the regulator system **310** as an "auditor" and the blockchain user **302** as a "client." An auditor could be restricted only to querying the ledger whereas a client could be authorized to deploy, invoke, and query certain types of chaincode.

A blockchain developer system **316** writes chaincode and client-side applications. The blockchain developer system **316** can deploy chaincode directly to the network through a REST interface. To include credentials from a traditional data source **330** in chaincode, the developer system **316** could use an out-of-band connection to access the data. In this example, the blockchain user **302** connects to the network through a peer node **312**. Before proceeding with any transactions, the peer node **312** retrieves the user's enrollment and transaction certificates from the certificate authority **318**. In some cases, blockchain users must possess these digital certificates in order to transact on the permissioned blockchain network **310**. Meanwhile, a user attempting to drive chaincode may be required to verify their credentials on the traditional data source **330**. To confirm the user's authorization, chaincode can use an out-of-band connection to this data through a traditional processing platform **320**.

FIG. 4A illustrates a flow diagram **400** of an example method of zero-trust distributed voting and vote management on a blockchain networks, according to example embodiments. Referring to FIG. 4A, the method **400** may include one or more of the steps described below.

FIG. 4A illustrates a flow chart of an example method executed by the Vote Management Server (VMS) node **102** (see FIG. 1A). It should be understood that method **400** depicted in FIG. 4A may include additional operations and that some of the operations described therein may be removed and/or modified without departing from the scope of the method **400**. The description of the method **400** is also

13

made with reference to the features depicted in FIG. 1B for purposes of illustration. Particularly, the processor 204 of the VMS node 102 may execute some or all of the operations included in the method 400.

With reference to FIG. 4A, at block 412, the processor 204 may acquire live biometrics data from a camera of the user device running a voting application. At block 414, the processor 204 may compare the live biometrics data against biometrics data from a user ID pre-recorded on the blockchain. At block 416, the processor 204 may responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, receive a vote data from the user device. At block 418, the processor 204 may record the vote data along with the verification verdict and the user ID as an encrypted vote token on a blockchain ledger.

FIG. 4B illustrates a further flow diagram 450 of an example method of zero-trust distributed voting and vote management system using a blockchain network, according to example embodiments. Referring to FIG. 4B, the method 450 may also include one or more of the following steps. At block 452, the processor 204 may generate a QR code to be presented to a voting official device connected to the vote management server node over the blockchain network and running a verification application.

At block 454, the processor 204 may responsive to verification of the QR code, generate the vote token based on data received from a voting application running on the user device. At block 456, the processor 204 may encrypt the vote token by a key derived from the live user biometrics data. At block 458, the processor 204 may, execute a smart contract to encrypt and record the vote token on a blockchain ledger. At block 460, the processor 204 may acquire geo-location data from the user device and compare the geo-location data to geo-location data recorded on the blockchain ledger at registration. At block 462, the processor 204 may, responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, provide a voting ballot interface data to the user device.

At block 464, the processor 204 may receive the live biometrics data from a user device hashed and encrypted by a user device key generated using a random seed derived from a blockchain ledger. At block 466, the processor 204 may derive an NFT of the user ID from the blockchain ledger and to acquire the biometrics data from the NFT. At block 468, the processor 204 may record the vote data along with the verification verdict and the user ID as a voter NFT token.

FIG. 5A illustrates an example system 500 that includes a physical infrastructure 510 configured to perform various operations according to example embodiments. Referring to FIG. 5A, the physical infrastructure 510 includes a module 512 and a module 514. The module 514 includes a blockchain 520 and a smart contract 530 (which may reside on the blockchain 520), that may execute any of the operational steps 508 (in module 512) included in any of the example embodiments. The steps/operations 508 may include one or more of the embodiments described or depicted and may represent output or written information that is written or read from one or more smart contracts 530 and/or blockchains 520. The physical infrastructure 510, the module 512, and the module 514 may include one or more computers, servers, processors, memories, and/or wireless communication devices. Further, the module 512 and the module 514 may be a same module.

FIG. 5B illustrates an example smart contract configuration among contracting parties and a mediating server con-

14

figured to enforce the smart contract terms on the blockchain according to example embodiments. Referring to FIG. 5B, the configuration 550 may represent a communication session, an asset transfer session or a process or procedure that is driven by a smart contract 530 which explicitly identifies one or more user devices 552 and/or 556. The execution, operations and results of the smart contract execution may be managed by a server 554. Content of the smart contract 530 may require digital signatures by one or more of the entities 552 and 556 which are parties to the smart contract transaction. The results of the smart contract execution may be written to a blockchain 520 as a blockchain transaction. The smart contract 530 resides on the blockchain 520 which may reside on one or more computers, servers, processors, memories, and/or wireless communication devices.

The above embodiments for zero-trust distributed voting and vote management system may be implemented in hardware, in a computer program executed by a processor, in firmware, or in a combination of the above. A computer program may be embodied on a computer readable medium, such as a storage medium. For example, a computer program may reside in random access memory ("RAM"), flash memory, read-only memory ("ROM"), erasable programmable read-only memory ("EPROM"), electrically erasable programmable read-only memory ("EEPROM"), registers, hard disk, a removable disk, a compact disk read-only memory ("CD-ROM"), or any other form of storage medium known in the art.

An exemplary storage medium may be coupled to the processor such that the processor may read information from, and write information to, the storage medium. In the alternative, the storage medium may be integral to the processor. The processor and the storage medium may reside in an application specific integrated circuit ("ASIC"). In the alternative, the processor and the storage medium may reside as discrete components.

FIG. 6A illustrates a process 600 of a new block being added to a distributed ledger 630, according to example embodiments, and FIG. 6B illustrates contents of a block structure 650 for blockchain, according to example embodiments. Referring to FIG. 6A, clients (not shown) may submit transactions to blockchain nodes 621, 622, and/or 623. Clients may be instructions received from any source to enact activity on the blockchain 630. As an example, clients may be applications that act on behalf of a requester, such as a device, person or entity to propose transactions for the blockchain. The plurality of blockchain peers (e.g., blockchain nodes 621, 622, and 623) may maintain a state of the blockchain network and a copy of the distributed ledger 630. Different types of blockchain nodes/peers may be present in the blockchain network including endorsing peers which simulate and endorse transactions proposed by clients and committing peers which verify endorsements, validate transactions, and commit transactions to the distributed ledger 630. In this example, the blockchain nodes 621, 622, and 623 may perform the role of endorser node, committer node, or both.

The distributed ledger 630 includes a blockchain 632 which stores immutable, voting sequence records in blocks, and a state database 634 (current world state) maintaining a current state of the blockchain 632. One distributed ledger 630 may exist per channel and each peer maintains its own copy of the distributed ledger 630 for each channel of which they are a member. The blockchain 632 is a transaction log, structured as hash-linked blocks where each block contains a sequence of N transactions. Blocks may include various components such as shown in FIG. 6B. The linking of the

15

blocks (shown by arrows in FIG. 6A) may be generated by adding a hash of a prior block's header within a block header of a current block. In this way, all transactions on the blockchain 632 are sequenced and cryptographically linked together preventing tampering with blockchain data without breaking the hash links. Furthermore, because of the links, the latest block in the blockchain 632 represents every transaction that has come before it. The blockchain 632 may be stored on a peer file system (local or attached storage), which supports an append-only blockchain workload.

The current state of the blockchain 632 and the distributed ledger 632 may be stored in the state database 634. Here, the current state data represents the latest values for all keys ever included in the chain transaction log of the blockchain 632. Chaincode invocations execute transactions against the current state in the state database 634. To make these chaincode interactions extremely efficient, the latest values of all keys are stored in the state database 634. The state database 634 may include an indexed view into the transaction log of the blockchain 632, it can therefore be regenerated from the chain at any time. The state database 634 may automatically get recovered (or generated if needed) upon peer startup, before transactions are accepted.

Endorsing nodes receive transactions from clients and endorse the transaction based on simulated results. Endorsing nodes hold smart contracts which simulate the transaction proposals. When an endorsing node endorses a transaction, the endorsing node creates a transaction endorsement which is a signed response from the endorsing node to the client application indicating the endorsement of the simulated transaction. The method of endorsing a transaction depends on an endorsement policy which may be specified within chaincode. An example of an endorsement policy is "the majority of endorsing peers must endorse the transaction." Different channels may have different endorsement policies. Endorsed transactions are forward by the client application to ordering service 610.

The ordering service 610 accepts endorsed transactions, orders them into a block, and delivers the blocks to the committing peers. For example, the ordering service 610 may initiate a new block when a threshold of transactions has been reached, a timer times out, or another condition. In the example of FIG. 6A, blockchain node 622 is a committing peer that has received a new data block 650 for storage on blockchain 630.

The ordering service 610 may be made up of a cluster of orderers. The ordering service 610 does not process transactions, smart contracts, or maintain the shared ledger. Rather, the ordering service 610 may accept the endorsed transactions and specifies the order in which those transactions are committed to the distributed ledger 630. The architecture of the blockchain network may be designed such that the specific implementation of 'ordering' (e.g., Solo, Kafka, BFT, etc.) becomes a pluggable component.

Transactions are written to the distributed ledger 630 in a consistent order. The order of transactions is established to ensure that the updates to the state database 634 are valid when they are committed to the network. Unlike a cryptocurrency blockchain system (e.g., Bitcoin, etc.) where ordering occurs through the solving of a cryptographic puzzle, or mining, in this example the parties of the distributed ledger 630 may choose the ordering mechanism that best suits that network.

When the ordering service 610 initializes a new block 650, the new block 650 may be broadcast to committing peers (e.g., blockchain nodes 621, 622, and 623). In response, each committing peer validates the transaction

16

within the new block 650 by checking to make sure that the read set and the write set still match the current world state in the state database 634. Specifically, the committing peer can determine whether the read data that existed when the endorsers simulated the transaction is identical to the current world state in the state database 634. When the committing peer validates the transaction, the transaction is written to the blockchain 632 on the distributed ledger 630, and the state database 634 is updated with the write data from the read-write set. If a transaction fails, that is, if the committing peer finds that the read-write set does not match the current world state in the state database 634, the transaction ordered into a block will still be included in that block, but it will be marked as invalid, and the state database 634 will not be updated.

Referring to FIG. 6B, a block 650 (also referred to as a data block) that is stored on the blockchain 632 of the distributed ledger 630 may include multiple data segments such as a block header 660, block data 670, and block metadata 680. It should be appreciated that the various depicted blocks and their contents, such as block 650 and its contents. Shown in FIG. 6B are merely for purposes of example and are not meant to limit the scope of the example embodiments. In some cases, both the block header 660 and the block metadata 680 may be smaller than the block data 670 which stores transaction data, however, this is not a requirement. The block 650 may store transactional information of N transactions (e.g., 100, 500, 1000, 2000, 3000, etc.) within the block data 670. The block 650 may also include a link to a previous block (e.g., on the blockchain 632 in FIG. 6A) within the block header 660. In particular, the block header 660 may include a hash of a previous block's header. The block header 660 may also include a unique block number, a hash of the block data 670 of the current block 650, and the like. The block number of the block 650 may be unique and assigned in an incremental/sequential order starting from zero. The first block in the blockchain may be referred to as a genesis block which includes information about the blockchain, its members, the data stored therein, etc.

The block data 670 may store transactional information of each transaction that is recorded within the block 650. For example, the transaction data may include one or more of a type of the transaction, a version, a timestamp, a channel ID of the distributed ledger 630, a transaction ID, an epoch, a payload visibility, a chaincode path (deploy tx), a chaincode name, a chaincode version, input (chaincode and functions), a client (creator) identify such as a public key and certificate, a signature of the client, identities of endorsers, endorser signatures, a proposal hash, chaincode events, response status, namespace, a read set (list of key and version read by the transaction, etc.), a write set (list of key and value, etc.), a start key, an end key, a list of keys, a Merkel tree query summary, and the like. The transaction data may be stored for each of the N transactions.

In some embodiments, the block data 670 may also store data 672 which adds additional information to the hash-linked chain of blocks in the blockchain 632. Accordingly, the data 672 can be stored in an immutable log of blocks on the distributed ledger 630. Some of the benefits of storing such data 672 are reflected in the various embodiments disclosed and depicted herein.

The block metadata 680 may store multiple fields of metadata (e.g., as a byte array, etc.). Metadata fields may include signature on block creation, a reference to a last configuration block, a transaction filter identifying valid and invalid transactions within the block, last offset persisted of

17

an ordering service that ordered the block, and the like. The signature, the last configuration block, and the orderer metadata may be added by the ordering service **610**. Meanwhile, a committer of the block (such as blockchain node **622**) may add validity/invalidity information based on an endorsement policy, verification of read/write sets, and the like. The transaction filter may include a byte array of a size equal to the number of transactions in the block data **670** and a validation code identifying whether a transaction was valid/invalid.

FIG. 7 is not intended to suggest any limitation as to the scope of use or functionality of embodiments of the application described herein. Regardless, the computing node **700** is capable of being implemented and/or performing any of the functionality set forth hereinabove.

In computing node **700** there is a computer system/server **702** (such as the Vote Management Server **102** described above), which is operational with numerous other general purposes or special purpose computing system environments or configurations. Examples of well-known computing systems, environments, and/or configurations that may be suitable for use with computer system/server **702** include, but are not limited to, personal computer systems, server computer systems, thin clients, thick clients, hand-held or laptop devices, multiprocessor systems, microprocessor-based systems, set top boxes, programmable consumer electronics, network PCs, minicomputer systems, mainframe computer systems, and distributed cloud computing environments that include any of the above systems or devices, and the like.

Computer system/server **702** may be described in the general context of computer system-executable instructions, such as program modules, being executed by a computer system. Generally, program modules may include routines, programs, objects, components, logic, data structures, and so on that perform particular tasks or implement particular abstract data types. Computer system/server **702** may be practiced in distributed cloud computing environments where tasks are performed by remote processing devices that are linked through a communications network. In a distributed cloud computing environment, program modules may be located in both local and remote computer system storage media including memory storage devices.

As shown in FIG. 7, computer system/server **702** in cloud computing node **700** is shown in the form of a general-purpose computing device. The components of computer system/server **702** may include, but are not limited to, one or more processors or processing units **704**, a system memory **706**, and a bus that couples various system components including system memory **706** to processor **704**.

The bus represents one or more of any of several types of bus structures, including a memory bus or memory controller, a peripheral bus, an accelerated graphics port, and a processor or local bus using any of a variety of bus architectures. By way of example, and not limitation, such architectures include Industry Standard Architecture (ISA) bus, Micro Channel Architecture (MCA) bus, Enhanced ISA (EISA) bus, Video Electronics Standards Association (VESA) local bus, and Peripheral Component Interconnects (PCI) bus.

Computer system/server **702** typically includes a variety of computer system readable media. Such media may be any available media that is accessible by computer system/server **702**, and it includes both volatile and non-volatile media, removable and non-removable media. System memory **706**, in one embodiment, implements the flow diagrams of the other figures. The system memory **706** can include computer

18

system readable media in the form of volatile memory, such as random-access memory (RAM) **710** and/or cache memory **712**. Computer system/server **702** may further include other removable/non-removable, volatile/non-volatile computer system storage media. By way of example only, storage system **714** can be provided for reading from and writing to a non-removable, non-volatile magnetic media (not shown and typically called a “hard drive”). Although not shown, a magnetic disk drive for reading from and writing to a removable, non-volatile magnetic disk (e.g., a “floppy disk”), and an optical disk drive for reading from or writing to a removable, non-volatile optical disk such as a CD-ROM, DVD-ROM or other optical media can be provided. In such instances, each can be connected to the bus by one or more data media interfaces. As will be further depicted and described below, memory **806** may include at least one program product having a set (e.g., at least one) of program modules that are configured to carry out the functions of various embodiments of the application.

Program/utility **716**, having a set (at least one) of program modules **718**, may be stored in memory **706** by way of example, and not limitation, as well as an operating system, one or more application programs, other program modules, and program data. Each of the operating system, one or more application programs, other program modules, and program data or some combination thereof, may include an implementation of a networking environment. Program modules **718** generally carry out the functions and/or methodologies of various embodiments of the application as described herein.

As will be appreciated by one skilled in the art, aspects of the present application may be embodied as a system, method, or computer program product. Accordingly, aspects of the present application may take the form of an entirely hardware embodiment, an entirely software embodiment (including firmware, resident software, micro-code, etc.) or an embodiment combining software and hardware aspects that may all generally be referred to herein as a “circuit,” “module” or “system.” Furthermore, aspects of the present application may take the form of a computer program product embodied in one or more computer readable medium(s) having computer readable program code embodied thereon.

Computer system/server **702** may also communicate with one or more external devices **720** such as a keyboard, a pointing device, a display **722**, etc.; one or more devices that enable a user to interact with computer system/server **702**; and/or any devices (e.g., network card, modem, etc.) that enable computer system/server **702** to communicate with one or more other computing devices. Such communication can occur via I/O interfaces **724**. Still yet, computer system/server **702** can communicate with one or more networks such as a local area network (LAN), a general wide area network (WAN), and/or a public network (e.g., the Internet) via network adapter **726**. As depicted, network adapter **726** communicates with the other components of computer system/server **702** via a bus. It should be understood that although not shown, other hardware and/or software components could be used in conjunction with computer system/server **702**. Examples include, but are not limited to: micro-code, device drivers, redundant processing units, external disk drive arrays, RAID systems, tape drives, and data archival storage systems, etc.

Although an exemplary embodiment of at least one of a system, method, and non-transitory computer readable medium has been illustrated in the accompanied drawings and described in the foregoing detailed description, it will be

understood that the application is not limited to the embodiments disclosed, but is capable of numerous rearrangements, modifications, and substitutions as set forth and defined by the following claims. For example, the capabilities of the system of the various figures can be performed by one or more of the modules or components described herein or in a distributed architecture and may include a transmitter, recipient or pair of both. For example, all or part of the functionality performed by the individual modules, may be performed by one or more of these modules. Further, the functionality described herein may be performed at various times and in relation to various events, internal or external to the modules or components. Also, the information sent between various modules can be sent between the modules via at least one of: a data network, the Internet, a voice network, an Internet Protocol network, a wireless device, a wired device and/or via plurality of protocols. Also, the messages sent or received by any of the modules may be sent or received directly and/or via one or more of the other modules.

One skilled in the art will appreciate that a "system" could be embodied as a personal computer, a server, a console, a personal digital assistant (PDA), a cell phone, a tablet computing device, a Smart phone or any other suitable computing device, or combination of devices. Presenting the above-described functions as being performed by a "system" is not intended to limit the scope of the present application in any way but is intended to provide one example of many embodiments. Indeed, methods, systems and apparatuses disclosed herein may be implemented in localized and distributed forms consistent with computing technology.

It should be noted that some of the system features described in this specification have been presented as modules, in order to more particularly emphasize their implementation independence. For example, a module may be implemented as a hardware circuit comprising custom very large-scale integration (VLSI) circuits or gate arrays, off-the-shelf semiconductors such as logic chips, transistors, or other discrete components. A module may also be implemented in programmable hardware devices such as field programmable gate arrays, programmable array logic, programmable logic devices, graphics processing units, or the like.

A module may also be at least partially implemented in software for execution by various types of processors. An identified unit of executable code may, for instance, comprise one or more physical or logical blocks of computer instructions that may, for instance, be organized as an object, procedure, or function. Nevertheless, the executables of an identified module need not be physically located together but may comprise disparate instructions stored in different locations which, when joined logically together, comprise the module and achieve the stated purpose for the module. Further, modules may be stored on a computer-readable medium, which may be, for instance, a hard disk drive, flash device, random access memory (RAM), tape, or any other such medium used to store data.

Indeed, a module of executable code could be a single instruction, or many instructions, and may even be distributed over several different code segments, among different programs, and across several memory devices. Similarly, operational data may be identified and illustrated herein within modules and may be embodied in any suitable form and organized within any suitable type of data structure. The operational data may be collected as a single data set or may be distributed over different locations including over differ-

ent storage devices, and may exist, at least partially, merely as electronic signals on a system or network.

It will be readily understood that the components of the application, as generally described and illustrated in the figures herein, may be arranged and designed in a wide variety of different configurations. Thus, the detailed description of the embodiments is not intended to limit the scope of the application as claimed but is merely representative of selected embodiments of the application.

One having ordinary skill in the art will readily understand that the above may be practiced with steps in a different order, and/or with hardware elements in configurations that are different than those which are disclosed. Therefore, although the application has been described based upon these preferred embodiments, it would be apparent to those of skill in the art that certain modifications, variations, and alternative constructions would be apparent.

While preferred embodiments of the present application have been described, it is to be understood that the embodiments described are illustrative only and the scope of the application is to be defined solely by the appended claims when considered with a full range of equivalents and modifications (e.g., protocols, hardware devices, software platforms, etc.) thereto.

What is claimed is:

1. A system for an automated zero-trust distributed voting and vote management based on protected voter-related data, comprising:

a processor of a vote management server node connected over a blockchain network to at least one user device onboarded onto the blockchain; and

a memory on which are stored machine-readable instructions that when executed by the processor, cause the processor to:

acquire live biometrics data from a camera of the user device running a voting application;

compare the live biometrics data against biometrics data from a user ID pre-recorded on the blockchain;

responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, receive a vote data from the user device; and

record the vote data along with the verification verdict and the user ID as an encrypted vote token on a blockchain ledger.

2. The system of claim 1, wherein the machine-readable instructions when executed by the processor, cause the processor to generate a QR code to be presented to a voting official device connected to the vote management server node over the blockchain network and running a verification application.

3. The system of claim 2, wherein the machine-readable instructions when executed by the processor, cause the processor to, responsive to verification of the QR code, generate the vote token based on data received from a voting application running on the user device.

4. The system of claim 1, wherein the machine-readable instructions when executed by the processor, cause the processor to encrypt the vote token by a key derived from the live user biometrics data.

5. The system of claim 1, wherein the machine-readable instructions when executed by the processor, cause the processor to execute a smart contract to encrypt and record the vote token on a blockchain ledger.

6. The system of claim 1, wherein the machine-readable instructions when executed by the processor, cause the processor to acquire geo-location data from the user device

21

and compare the geo-location data to geo-location data recorded on the blockchain ledger at registration.

7. The system of claim 1, wherein the machine-readable instructions when executed by the processor, cause the processor to, responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, provide a voting ballot interface data to the user device.

8. The system of claim 1, wherein the machine-readable instructions when executed by the processor, cause the processor to receive the live biometrics data from a user device hashed and encrypted by a user device key generated using a random seed derived from a blockchain ledger.

9. The system of claim 1, wherein the machine-readable instructions when executed by the processor, cause the processor to derive an NFT of the user ID from the blockchain ledger and to acquire the biometrics data from the NFT.

10. The system of claim 1, wherein the machine-readable instructions when executed by the processor, cause the processor to record the vote data along with the verification verdict and the user ID as a voter NFT token.

11. A method for an automated zero-trust distributed voting and vote management based on protected voter-related data, comprising:

acquiring, by a vote management server (VMS) node, live biometrics data from a camera of a user device running a voting application;

comparing, by the VMS node, the live biometrics data against biometrics data from a user ID pre-recorded on the blockchain;

responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, receiving a vote data from the user device; and

recording, by the VMS node, the vote data along with the verification verdict and the user ID as an encrypted vote token on a blockchain ledger.

12. The method of claim 11, further comprising generating a QR code to be presented to a voting official device connected to the vote management server node over the blockchain network and running a verification application.

22

13. The method of claim 12, further comprising, responsive to verification of the QR code, generating the vote token based on data received from a voting application running on the user device.

14. The method of claim 11, further comprising encrypting the vote token by a key derived from the live user biometrics data.

15. The method of claim 11, further comprising executing a smart contract to encrypt and record the vote token on a blockchain ledger.

16. The method of claim 11, further comprising acquiring geo-location data from the user device and comparing the geo-location data to geo-location data recorded on the blockchain ledger at registration.

17. The method of claim 11, further comprising, responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, providing a voting ballot interface data to the user device.

18. The method of claim 11, further comprising receiving the live biometrics data from a user device hashed and encrypted by a user device key generated using a random seed derived from a blockchain ledger.

19. The method of claim 11, further comprising deriving an NFT of the user ID from the blockchain ledger and acquiring the biometrics data from the NFT and recording the vote data along with the verification verdict and the user ID as a voter NFT token.

20. A non-transitory computer-readable medium comprising instructions, that when read by a processor, cause the processor to perform:

acquiring live biometrics data from a camera of a user device running a voting application;

comparing the live biometrics data against biometrics data from a user ID pre-recorded on the blockchain;

responsive to a positive verification verdict of the live biometrics data against biometrics data from the user ID, receiving a vote data from the user device; and

recording the vote data along with the verification verdict and the user ID as an encrypted vote token on a blockchain ledger.

* * * * *